MCP Registry Landscape

Comparison, SecureMCP Positioning and Business Use Cases

A non-technical comparison of how Claude Code, Codex and other MCP ecosystems discover and connect to MCP services, followed by practical use cases for a Secured MCP Registry.

Executive conclusion

1. **Claude Code —** connects to MCP servers that a user or team configures. Its documentation provides examples and points users toward community servers, but it does not describe Claude Code as automatically selecting servers from one mandatory registry.
2. **Codex —** also connects to explicitly configured MCP servers. Local Codex clients share MCP configuration, while hosted ChatGPT and Codex experiences can receive MCP-backed tools through plugins.
3. **Public registries —** primarily help people discover, publish, install or host MCP servers. Their focus ranges from open metadata to managed credentials, containers, gateways and operational monitoring.
4. **Secured MCP Registry —** is positioned as a governed control plane rather than only a directory. It combines publication and discovery with certification, policy, contracts, consent, provenance, adaptive risk controls and lifecycle actions.

Important terminology

Claude Code and Codex are MCP clients, not registries. A registry helps users discover or govern MCP servers; the client connects to the selected server after configuration or installation.

Therefore, the most accurate question is not “Which registry does Claude Code or Codex use?” but “How do these clients discover, configure and govern MCP servers?”

How Claude Code works with MCP

5. **Connection model —** Users add local or remote MCP servers through Claude Code commands or configuration.
6. **Team sharing —** Project-scoped server definitions can be stored in a shared .mcp.json file.
7. **User approval —** Claude Code asks for approval before using project-scoped MCP servers from shared configuration.
8. **Authentication —** Remote HTTP and SSE servers can use OAuth, headers or other configured credentials.

9. **Discovery —** Anthropic documentation lists popular servers and directs users to broader community sources, while installation remains an explicit user or team action.
10. **Security message —** Anthropic warns that third-party servers are not all verified and should be installed only when trusted.

How Codex works with MCP

11. **Connection model —** Codex supports local STDIO servers and remote Streamable HTTP servers.
12. **Shared configuration —** The ChatGPT desktop app, Codex CLI and Codex IDE extension share MCP configuration on the same Codex host.
13. **Project control —** Servers can be configured globally or in a trusted project through config.toml.
14. **Tool control —** Codex supports server enablement, required-server settings, tool allowlists, tool denylists and approval modes.
15. **Hosted tools —** ChatGPT and Codex can also receive remote MCP-backed tools through installed plugins rather than local MCP configuration.
16. **Discovery —** Official Codex documentation provides examples of useful MCP servers, but does not describe a built-in public-registry search command comparable to GitHub Copilot CLI's registry search.

Registry and catalogue comparison

1. Official MCP Registry

Primary role: A community-driven central repository and API for publishing and discovering MCP server metadata.

Strength: Open ecosystem metadata, standard registry interfaces and interoperability.

Boundary: It is currently described as preview. Published version metadata is immutable, and its FAQ directs malicious-server reports to the underlying package registry and a GitHub issue.

SecureMCP relevance: Best used as an ecosystem source of record or upstream feed, not by itself as a complete enterprise runtime-security system.

2. GitHub MCP Registry

Primary role: A discovery experience for finding official or relevant MCP servers within GitHub-oriented developer workflows.

Strength: Fast discovery and installation, plus enterprise support for internal registries and allowlist controls in supported GitHub environments.

Boundary: Its strongest value is developer workflow integration and organizational distribution policy.

SecureMCP relevance: A useful comparison for self-service discovery, but SecureMCP goes further into execution-time governance and adaptive risk.

3. Docker MCP Catalog and Toolkit

Primary role: A curated catalogue designed to make MCP servers easy to discover, deploy and run with Docker tooling.

Strength: Container packaging, repeatable deployment and desktop-based operational convenience.

Boundary: It is especially strong when the main problem is safely packaging and running third-party server software.

SecureMCP relevance: SecureMCP can complement this model by governing which containerized server may be used, by whom and under which policy.

4. Smithery

Primary role: An open registry combined with managed MCP connections, OAuth, token refresh and credential storage.

Strength: Low-friction connection management for agent builders and multi-user applications.

Boundary: It reduces integration effort by managing authentication and connection lifecycle.

SecureMCP relevance: SecureMCP differentiates through organization-owned policy, certification, consent, contracts, provenance and reflexive controls.

5. Glama

Primary role: A large MCP directory with hosting, deployment, gateway, logs, analytics and per-tool access controls.

Strength: Discovery plus production operations, health checks, hosting and connection-profile security.

Boundary: It combines a public directory with operational infrastructure and runtime visibility.

SecureMCP relevance: It is the closest operational comparison, while SecureMCP emphasizes an integrated governance model across the full trust lifecycle.

6. PureCipher Secured MCP Registry

Primary role: A governed registry and security control plane built around SecureMCP.

Strength: Publisher verification, certification, policy, contracts, consent, provenance, Reflexive Core, alerts, moderation and lifecycle control.

Boundary: It is intended to make trust decisions before publication, during discovery and while an MCP capability is running.

SecureMCP relevance: Its clearest position is “governed MCP adoption for organizations,” rather than “the largest public directory.”

Comparison at a glance

Option	Best suited to	Discovery model	Governance emphasis
Official MCP Registry	Open ecosystem metadata	Public API and registry	Publisher metadata and ecosystem reporting
GitHub MCP Registry	Developer self-service	GitHub-integrated search	Enterprise allowlists in supported GitHub environments
Docker MCP Catalog	Packaged deployment	Curated Docker catalogue	Container execution and operational consistency
Smithery	Managed connections	Open registry and connection API	OAuth, credentials and session lifecycle
Glama	Hosted MCP operations	Directory plus deployment	Gateway, logs, health and per-tool controls
Secured MCP Registry	Governed organizational adoption	Approved internal/public catalogue	Policy, consent, contracts, provenance and adaptive risk

Where the Secured MCP Registry is different

17. **Governance before discovery —** Only approved publishers, services and versions should become visible to the intended audience.
18. **Policy during execution —** A service can be discoverable but still restricted for a particular user, agent, purpose or environment.
19. **Consent and contracts —** Access can depend on explicit permission and an agreed purpose rather than on installation alone.
20. **Traceable provenance —** Important requests, decisions and outcomes can be recorded for investigation and accountability.
21. **Adaptive protection —** The Reflexive Core can detect behavioural drift and escalate from normal operation to approval, restriction or blocking.
22. **Lifecycle enforcement —** Moderators can approve, suspend, deprecate, revoke or deregister services and versions.
23. **Client identity —** AI agents, services and frameworks can receive controlled identities and tokens instead of appearing as anonymous consumers.

24. **Enterprise deployment choice —** The registry can be operated as an internal catalogue, a curated public service or a federated trust environment.

Business use cases

1. Enterprise-approved AI tools

Situation: Employees want Claude Code, Codex or another AI client to use Jira, GitHub, databases and internal APIs.

Registry response: The organization publishes an approved catalogue and blocks or discourages unreviewed MCP servers.

Key pillars: Registry, identity, certification, policy and client tokens.

2. Regulated financial-services agent

Situation: An AI agent prepares a customer or transaction analysis using sensitive systems.

Registry response: Consent, purpose limits and audit evidence ensure that the agent accesses only approved data for the approved task.

Key pillars: Policy, contracts, consent, provenance, compliance and alerts.

3. Healthcare workflow assistant

Situation: A clinical or administrative assistant needs controlled access to patient-related services.

Registry response: The registry approves specific service versions, while runtime controls limit access by role, purpose and environment.

Key pillars: Certification, policy, consent, sandboxing, provenance and revocation.

4. Third-party MCP vendor onboarding

Situation: A SaaS vendor wants its MCP service to be available to enterprise AI agents.

Registry response: The publisher submits identity, capability, permission and version evidence; reviewers approve or request remediation.

Key pillars: Publisher service, certification, moderation, registry and notifications.

5. Developer self-service with guardrails

Situation: Engineering teams want easy access to documentation, observability, browser and repository tools.

Registry response: Developers discover pre-approved servers without waiting for a manual installation review each time.

Key pillars: Catalogue, install guidance, policy profiles and client identity.

6. Secure API-to-MCP conversion

Situation: An organization wants to expose selected internal REST operations to AI agents.

Registry response: OpenAPI operations are converted into governed MCP toolsets while credentials remain controlled.

Key pillars: OpenAPI ingestion, gateway, policy, contracts, provenance and alerts.

7. High-risk production action

Situation: An AI agent requests a deployment, payment, account change or destructive infrastructure action.

Registry response: The request is checked against policy and consent, executed in a restricted environment where appropriate and stopped when behaviour becomes abnormal.

Key pillars: Policy, consent, contracts, sandbox, Reflexive Core and audit.

8. Rapid incident response

Situation: A trusted server version or publisher credential becomes compromised.

Registry response: Administrators revoke trust, suspend the listing, notify affected clients and prevent new connections.

Key pillars: Revocation, registry lifecycle, alerts, client inventory and provenance.

9. Multi-agent workflow control

Situation: A coordinator agent delegates work to specialist agents that use different MCP services.

Registry response: Each agent receives a distinct identity and only the tools required for its assigned role.

Key pillars: Client identity, policy, contracts, consent and provenance.

10. Federated organization

Situation: Business units or partner organizations want to share approved MCP services without surrendering local governance.

Registry response: Trust information can be exchanged while each organization keeps its own approval and policy decisions.

Key pillars: Federation, registry, certification, revocation and local policy.

11. Controlled version promotion

Situation: A publisher needs to move an MCP service from development to testing and production.

Registry response: Each version is reviewed, certified and promoted according to environment-specific policy.

Key pillars: Policy versioning, certification, moderation, provenance and notifications.

12. Shadow MCP visibility

Situation: Teams are using untracked MCP servers with unknown permissions or credentials.

Registry response: The organization establishes a registered client inventory and an approved catalogue, making exceptions visible and reviewable.

Key pillars: Client registry, catalogue, policy, audit and alerts.

Recommended positioning

25. **Do not compete on catalogue size alone —** Public registries and directories are already optimized for broad discovery.
26. **Use public registries as upstream sources —** Import or reference ecosystem metadata, then apply organizational approval and security enrichment.
27. **Lead with governed adoption —** Position the Secured MCP Registry as the place where an organization decides which MCP services and versions may be trusted.
28. **Emphasize runtime protection —** The strongest distinction is the combination of policy, consent, contracts, provenance and the Reflexive Core after installation.
29. **Support multiple clients —** The registry should serve Claude Code, Codex and any standards-compliant MCP client rather than becoming tied to one model vendor.

Sources

30. **Anthropic: Connect Claude Code to tools via MCP —** https://docs.anthropic.com/en/docs/claude-code/mcp

31. **OpenAI Codex Manual: Model Context Protocol —** https://learn.chatgpt.com/docs/extend/mcp
32. **Official MCP Registry API —** https://registry.modelcontextprotocol.io/docs
33. **Official MCP Registry FAQ —** https://modelcontextprotocol.io/registry/faq
34. **GitHub: Meet the GitHub MCP Registry —** https://github.blog/ai-and-ml/github-copilot/meet-the-github-mcp-registry-the-fastest-way-to-discover-mcp-servers/
35. **GitHub: Internal MCP registry and allowlist controls —** https://github.blog/changelog/2025-09-12-internal-mcp-registry-and-allowlist-controls-for-vs-code-insiders/
36. **Docker MCP Registry —** https://github.com/docker/mcp-registry
37. **Smithery documentation —** https://smithery.ai/docs
38. **Smithery Connect —** https://smithery.ai/docs/use/connect
39. **Glama MCP hosting and gateway —** https://glama.ai/mcp/hosting

Research note: Public product capabilities change quickly. This comparison reflects official documentation reviewed on 28 July 2026. Statements about the PureCipher Secured MCP Registry are based on the current project implementation and architecture.